

Documentazione Pellegrini - 635452

TRIVIA QUIZ MULTIPLAYER

Il progetto implementa il gioco "Trivia Quiz Multiplayer", una forma di intrattenimento interattiva che stimola i partecipanti a rispondere alle domande inviate dal server, con l'obiettivo di ottenere il maggior numero di risposte corrette e posizionarsi più in alto in classifica. I temi disponibili sono 3: Capitali e Geografia, Squadre e Giocatori e Videogiochi classici.

Ognuno dei 3 temi è composto da 5 domande. Il server mantiene una classifica per ciascun tema in base al punteggio realizzato. Le domande e le risposte sono estratte da due file di testo: ``questions.txt`` e ``answers.txt``, che devono essere salvati nella stessa directory dell'eseguibile. Il file delle domande contiene le 5 domande del primo tema, le 5 del secondo e le 5 del terzo, mentre quello delle risposte contiene l'ordine corrispondente delle risposte.

Per caricare questi dati, il progetto usa un array `'quizzes'`, di dimensione pari al prodotto tra temi e domande per tema, di tipo struct `'quiz'`. In particolare, la struct contiene al suo interno la domanda e la relativa risposta. Il server, al momento dell'avvio, legge queste domande e risposte dai due files e le memorizza per poi poter estrarre velocemente i quesiti.

Tra client e server i dati vengono scambiati tramite socket. La scelta del protocollo di trasporto è TCP, in quanto abbiamo bisogno di un trasferimento dati affidabile (garantisce l'ordine e l'integrità dei pacchetti) e di una connessione persistente senza la necessità di avere un gameplay super veloce.

Per lo scambio dei messaggi sono state definite due funzioni, chiamate ``send_msg`` e ``recv_msg``. Queste funzioni si occupano inizialmente di inviare/ricevere un TAG di dimensione fissa, utile per la gestione delle varie fasi di gioco, poi la lunghezza del messaggio, ed infine il contenuto vero e proprio del messaggio.

Questa scelta aumenta leggermente il traffico, ma rende più semplice l'implementazione, perché non c'è necessità di definire strutture rigide per ogni tipo di messaggio. Poiché la maggior parte dei dati trasmessi sono stringhe, ho scelto un protocollo testuale personalizzato che risulta anche più leggibile, facile da debuggare e più intuitivo rispetto a una struttura binaria.

Ho deciso di introdurre un tag, oltre allo scambio della lunghezza del messaggio prima del messaggio vero e proprio per fare in modo di constatare in quale fase del gioco il client si trova (e quindi l'azione giusta che il server deve fare), così da facilitare lo sviluppo dell'applicazione.

SERVER

Il server si compila tramite il comando `'gcc -Wall -o server server_pellegrini.c'` e si esegue semplicemente con `'./server'`.

Per la gestione delle connessioni, il server utilizza l'I/O multiplexing per gestire connessioni multiple su socket in modo non bloccante e senza multithreading, tramite la funzione `select`. In particolare viene definito un set su cui viene applicata la `select`, all'interno del quale vengono monitorati i client che richiedono la connessione al server e che sono connessi.

Quando viene recapitata una richiesta di connessione al server, viene mandata in esecuzione la funzione `'new_connection'`, la quale fa sì che il server accetti la richiesta di connessione (se c'è posto disponibile) ed inizializzi un nuovo giocatore (memorizza il socket relativo nella struttura dati `players`) nel primo elemento disponibile dell'array `"players"`. Successivamente, invia al client appena connesso la richiesta di nickname.

Attraverso l'uso di set di descrittori e della `select`, il server controlla anche quando un client che si era precedentemente connesso invia un messaggio. Se ciò accade, la funzione `'manage_connection'` gestisce la partita del giocatore: la sua esecuzione cambia a seconda della fase dell'applicazione, appresa tramite la ricezione di un tag di dimensione fissa insieme al

messaggio. Inoltre, la stessa funzione, gestisce i casi in cui il giocatore voglia visionare la classifica tramite il comando "show score" e in cui voglia uscire dal quiz tramite "endquiz".

Ogni volta, dopo aver gestito un client, il server azzera il set e il max_sd prima di ogni chiamata a select() in quanto quest'ultima modifica i set passati, quindi senza questo passaggio non verranno controllati più i socket giusti. Dopo il reset, vengono aggiunti di nuovo tutti i socket da monitorare controllando la struttura dati dedicata a ogni giocatore.

Quando un giocatore completa un quiz, il server aggiorna i punteggi e, se il giocatore ha terminato tutti i quiz, può inviargli la schermata finale con la classifica.

Viene gestita la chiusura del terminale del server e l'esecuzione di `ctrl^C` tramite l'uso di handler dei segnali. Se il server riceve SIGINT o SIGHUP:

- Chiude tutti i socket dei client connessi,
- Chiude il socket del server,
- Termina l'esecuzione.

Quando i client proveranno a scrivere successivamente sul socket chiuso, riceveranno un errore e sapranno che il server non è più disponibile.

Il server è implementato per gestire fino a 10 client (impostato da `MAX\_PLAYERS`) contemporaneamente. Se l'undicesimo client prova a connettersi, il server rifiuta la connessione e comunica al client l'esaurimento di posti disponibili.

CLIENT

Il client si compila con il comando 'gcc -Wall -o client client_pellegrini.c' e si esegue con './client [porta]' dove '[porta]' è opzionale (di default è 1234).

All'avvio, il client mostra un piccolo menù testuale dove l'utente può decidere se avviare una sessione di Trivia o uscire. Se l'utente sceglie `1`, il client prova a connettersi al server (richiesta di `connect`). Se la connessione va a buon fine, il server chiederà il nickname. La richiesta del nickname continua ad essere fatta dal server finché il nuovo client connesso non specifica un nickname che non è già stato scelto da nessun client.

Una volta connesso, il client entra in un loop in cui:

1. Riceve un messaggio dal server (`recv\_msg`) che contiene un tag e un payload (ad esempio una domanda di quiz, oppure il menù di scelta tema).
2. Stampa il messaggio a schermo per l'utente.
3. L'utente inserisce da tastiera la propria risposta (o comando speciale come "show score" o "endquiz").
4. Il client invia la risposta al server (`send\_msg`) col tag appropriato a seconda della fase di gioco (es. `ANSWER`, `MENU\_TAG`, `QUIT\_TAG`).

Un giocatore può inviare alcuni comandi speciali, dopo l'inizio di una partita (avvio del primo quiz):

- 'show score' per richiedere la classifica aggiornata. Il server invierà la classifica e attenderà la risposta all'ultima domanda che ha fatto, così che l'utente possa rispondere effettivamente a quella domanda o scegliere il tema.
- 'endquiz' per abbandonare la partita. Il server rimuove i dati del giocatore, chiude la connessione e il client torna al menù di avvio.

Quando il giocatore completa tutti e tre i quiz, il server manderà un messaggio finale con la classifica e attenderà un comando "endquiz" o "show score". Se il client risponde con 'endquiz' ritornerà al piccolo menù iniziale, mentre il server rilascerà le risorse associate al giocatore.

Lato client, se si preme `ctrl^C` o si chiude il terminale, viene inviato lo stesso tag (QUIT_TAG) di "endquiz" al server, in modo che il server liberi immediatamente le risorse associate a quel giocatore. Il client poi chiude il suo socket e termina.